

EX 1 THREE-LAYER NEURAL NETWORK FROM SCRATCH

Aim:

To implement a three-layer neural network from scratch using Python and backpropagation for handwritten digit recognition using the MNIST dataset.

Algorithm:

1. Import NumPy, Matplotlib, TensorFlow MNIST, and OneHotEncoder.
2. Load the MNIST training dataset.
3. Reshape images from 28×28 into 784-element vectors.
4. Normalize pixel values by dividing by 255.
5. Convert digit labels into one-hot encoded vectors.
6. Initialize weights and biases for the input, hidden, and output layers.
7. Perform forward propagation using the sigmoid activation function.
8. Calculate cross-entropy loss and perform backpropagation.
9. Update weights and biases using gradient descent.
10. Repeat training for multiple epochs, plot the loss, and predict a sample digit.

Program

```
import numpy as np

import matplotlib.pyplot as plt

from tensorflow.keras.datasets import mnist

from sklearn.preprocessing import OneHotEncoder

# Load and preprocess data

(X_train, y_train), (_, _) = mnist.load_data()

X_train_images = X_train.copy()

X_train = X_train.reshape(-1, 784).astype("float32") / 255.0
```

```
encoder = OneHotEncoder(sparse_output=False)

y_train_onehot = encoder.fit_transform(y_train.reshape(-1, 1))

# Initialize network

input_size, hidden_size, output_size = 784, 64, 10

rng = np.random.default_rng(42)

W1 = rng.standard_normal((input_size, hidden_size)) * 0.01
b1 = np.zeros((1, hidden_size))

W2 = rng.standard_normal((hidden_size, output_size)) * 0.01
b2 = np.zeros((1, output_size))

# Activation and loss

def sigmoid(x):

    x = np.clip(x, -500, 500)

    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(a):

    return a * (1 - a)

def binary_cross_entropy(y, y_hat):

    eps = 1e-8

    return -np.mean(y * np.log(y_hat + eps) +

        (1 - y) * np.log(1 - y_hat + eps))
```

```

# Training

epochs, learning_rate = 10, 0.1

losses = []

for epoch in range(epochs):

    total_loss = 0

    for i in range(X_train.shape[0]):

        x = X_train[i:i+1]

        y = y_train_onehot[i:i+1]

        z1 = x @ W1 + b1

        a1 = sigmoid(z1)

        z2 = a1 @ W2 + b2

        a2 = sigmoid(z2)

        loss = binary_cross_entropy(y, a2)

        total_loss += loss

        dz2 = a2 - y

        dW2 = a1.T @ dz2

        db2 = dz2

        dz1 = (dz2 @ W2.T) * sigmoid_derivative(a1)

        dW1 = x.T @ dz1

```

```
db1 = dz1
```

```
W2 -= learning_rate * dW2
```

```
b2 -= learning_rate * db2
```

```
W1 -= learning_rate * dW1
```

```
b1 -= learning_rate * db1
```

```
average_loss = total_loss / X_train.shape[0]
```

```
losses.append(average_loss)
```

```
print(f'Epoch {epoch + 1}, Loss: {average_loss:.4f}')
```

```
# Plot loss
```

```
plt.plot(losses)
```

```
plt.title("Training Loss")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Loss")
```

```
plt.grid(True)
```

```
plt.show()
```

```
# Prediction
```

```
def predict(img):
```

```
    img = img.reshape(1, 784).astype("float32") / 255.0
```

```
    a1 = sigmoid(img @ W1 + b1)
```

```
    a2 = sigmoid(a1 @ W2 + b2)
```

```
    return np.argmax(a2)
```

```
idx = 100
```

```
predicted_digit = predict(X_train_images[idx])
```

```
plt.imshow(X_train_images[idx], cmap="gray")
```

```
plt.title(f"Prediction: {predicted_digit}, Actual: {y_train[idx]}")
```

```
plt.axis("off")
```

```
plt.show()
```

Output:

```
Epoch 1/10, Loss: 0.0448
```

```
Epoch 2/10, Loss: 0.0260
```

```
Epoch 3/10, Loss: 0.0213
```

```
Epoch 4/10, Loss: 0.0189
```

```
Epoch 5/10, Loss: 0.0166
```

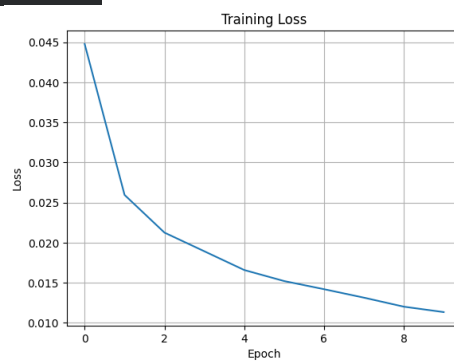
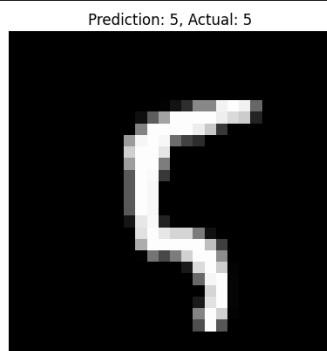
```
Epoch 6/10, Loss: 0.0152
```

```
Epoch 7/10, Loss: 0.0142
```

```
Epoch 8/10, Loss: 0.0131
```

```
Epoch 9/10, Loss: 0.0120
```

```
Epoch 10/10, Loss: 0.0113
```



Result

The three-layer neural network was successfully implemented and trained on MNIST. The loss decreased during training, and the model successfully predicted handwritten digits.

